YANTRA

Team Build Plan v2.0

Phase 2 — Performance · Practice Rooms · Multi-LLM · Production Hardening

---

Confidential · Yantra Team · 2025–2026 · yantra.build

5-Member Engineering Sprint Plan

THIS DOCUMENT COVERS WHAT IS LEFT TO BUILD. IT DOES NOT REPEAT WHAT IS ALREADY LIVE.

01 — Context & What Is Already Live

The following surfaces are already in production and are excluded from this plan:

Already Built	Status
Public marketing landing page (yantra.build)	Live
Email/password auth + Google OAuth via Supabase	Live
Onboarding flow — role, age, goals, pace	Live
Protected student dashboard (seed data)	Live
Student profile page + persistence	Live
Gemini-powered Yantra chat (widget + full panel)	Live
Authenticated chat history (rolling 40 msgs)	Live
Access request form + Supabase persistence	Live
Docs site with Support Desk AI	Live
Supabase schema — profiles, chat_histories, dashboard tables	Live
Route transition animations + Lenis smooth scroll	Live

Everything below is net-new work for the team sprint.

02 — Performance & Speed Optimisation

The current build is functional but unoptimised. Before adding features, the team must close the performance gap. This is the highest priority for the first sprint.

Metric	Current (estimate)	Target < 1.8 s <
LCP (Largest Contentful Paint)	> 3 s Unknown	0.05 < 150 ms <
CLS (Layout Shift)	Unknown	200 KB gzipped <
INP / FID	Unaudited Slow on	2 s on 4G
JS Bundle (initial)	mobile	
Dashboard TTI		

• Run Lighthouse audit on /, /dashboard, /onboarding — document baseline scores in #deployments. • Enable Next.js Image Optimization on all img tags — convert to next/image with explicit width/height. • Enable dynamic import() for heavy dashboard sections (ChatWidget, motion components) with loading skeletons. • Extract and defer non-critical CSS. KaTeX CSS loads on every page — lazy-load it only when markdown is rendered.

• Audit Framer Motion / motion bundle — replace page-level AnimatePresence wrappers with CSS transitions where motion adds no value.

• Enable Turbopack in dev (already in next.config.ts) — confirm it is not bleeding into prod bundle.

• Set up Vercel Edge caching headers on /api/chat/history and /api/profile (stale-while-revalidate).

• Enable Supabase connection pooling via PgBouncer (Supabase dashboard setting — free tier).

• Add React.memo() or useMemo() around the StudentDashboard skill and room card grids — they re-render on every chat open.

• Compress all static assets. Run next build and check .next/analyze (add @next/bundle-analyzer to devDeps).

• Test every page at 320px, 375px, 768px, 1024px, 1440px. Log broken layouts in #bugs.

• Fix the student profile page side-nav — it uses a fixed 264px aside that breaks at 1024–1280px.

• Fix the onboarding footer action bar — CTA buttons overflow on iPhone SE (320px).

• Fix the dashboard hero section — font-size [7rem] clips on Android Chrome at 360px.

• Fix ChatWidget panel — on mobile it covers the full screen but the Escape-to-close hint is desktop-only.

• All touch targets must be >= 44px. Audit the nav icon buttons and the chat send button.

03 — Practice Rooms (Voids) — Phase 1 Build

This is the core differentiator of Yantra. Phase 1 delivers two functional Voids. The other six ship in Phase 2.

Void	Phase	Owner	Tech
Python Void — Monaco + Pyodide runner + AI feedback	1 (Sprint 1)	TBD	Monaco, Pyodide WASM
Algorithm Void — step visualiser (sorting, search)	1 (Sprint 2)	TBD	Custom JS + SVG
Neural Net Builder — drag-drop layers, TF.js train	2	TBD	TensorFlow.js
Data Explorer — CSV load, Plotly charts, AI narration	2	TBD	Pandas (Pyodide), Plotly.js
Prompt Engineer — prompt test, score, iterate	2	TBD	Claude / Gemini API

Model Inspector — ONNX weights + activation viewer	2	TBD	ONNX.js
Project Builder — end-to-end AI app sandbox	2	TBD	Full stack sandbox
Deploy Void — serverless deploy, get live URL	2	TBD	Cloudflare Workers

- Route: /dashboard/void/python — protected, auth-gated.
- Monaco Editor instance (CDN import, lazy-loaded) with Python syntax highlighting.
- Pyodide WASM runner — load from CDN, execute user code in browser, stream stdout to output panel.
- AI feedback panel — on run, send code + output to /api/void/feedback (new route, Claude Haiku, < 150 token response).
- Challenge system — seed 5 Python challenges in Supabase. Unlock next on pass. Track in student_progress table (new).
- Mobile layout — editor stacks above output on < 768px. Touch keyboard safe area handled.
- Error handling — Pyodide load failure shows a clear message, not a blank screen.

Algorithm Void — Sprint 2 Spec

- Route: /dashboard/void/algorithm
- Custom SVG step-by-step visualiser for bubble sort, merge sort, binary search, BFS, DFS.
- Speed slider (steps per second). Play / pause / step-forward controls.
- AI explains each step on pause — lightweight Haiku call, streamed inline.
- No external heavy libraries. Pure JS + React state + SVG. Must load in < 1 s.

04 — Multi-LLM Orchestration Layer

Currently all AI calls go directly to Gemini Flash. The orchestration layer routes each request to the best and cheapest model. This reduces cost and improves response quality per task type.

Task	Model	Route / File app/api/chat/route.ts
Main Yantra chat	Gemini 2.5 Flash (current)	app/api/void/feedback/route.ts (new)
Void code feedback (fast)	Claude Haiku 4.5	orchestrator selects on complexity flag
Deep concept explanation	Claude Sonnet 4.6	app/api/quiz/grade/route.ts (new)
Skill diagnosis quiz grading	Gemini Flash (cheap)	app/api/search/route.ts (new)
Web search synthesis	Gemini + Serper.dev	app/api/docs-support/route.ts
Docs support (Support Desk)	Gemini 2.5 Flash (current)	

- Create src/lib/ai/orchestrator.ts — exports selectModel(task, complexity) returning model string + API key.
- Add ANTHROPIC_API_KEY to .env.example and Vercel environment. Wire into orchestrator.
- Add complexity scoring — short questions route to Haiku, multi-step proofs route to Sonnet. Simple regex on token estimate.
- Rate-limit each model tier separately using Upstash Redis free tier (10k requests/day free).
- Log model used per request to Supabase analytics table (user_id, model, tokens, task_type, created_at) for cost tracking.
- Fallback chain: if Sonnet fails → Gemini Flash. If Haiku fails → Gemini Flash. Never fail silently.

05 — Skill Diagnosis & Dynamic Roadmap

The current dashboard shows static skill cards. This sprint makes them reflect real student progress tracked in Supabase.

New Database Tables

Table	Purpose
student_skill_mastery	Tracks each student's % per skill key, updated after quiz + void completion
student_void_sessions	Logs each void session: start, end, challenges_passed, code_runs
student_quiz_results	Quiz ID, score, wrong answers, time taken, created_at
yantra_challenges	Seed table — 5 challenges per void, difficulty 1-5, expected output
yantra_quiz_bank	Questions per skill, correct answer, distractors, explanation

- POST /api/skill/update — called after every void session + quiz. Recalculates mastery % per skill.
- Mastery formula: (quizzes_passed × 0.4) + (void_challenges_passed × 0.6) — capped at 100.
- Dashboard skill cards now read from student_skill_mastery via /api/profile (extend existing route).
- Locked state is real — a skill unlocks only when prerequisite mastery >= 60%.
- Roadmap card on dashboard shows next_recommended_void based on lowest unlocked mastery score.

06 — Smartboard / Classroom Mode

Zero-install classroom interface. A teacher opens a URL on the smartboard, students follow on tablets or phones.

- Route: /classroom — public, no auth required for view, teacher auth for control.
- URL param: /classroom?topic=python-basics&mode;=demo renders fullscreen void in presentation mode.
- Teacher mode: /classroom/teacher — protected route, shows class progress grid (requires teacher role in profiles table).
- Add teacher_code field to profiles. Teacher signs up normally, then sets role = 'Teacher' in profile. Dashboard shows teacher UI.
- Fullscreen kiosk CSS — hide nav, remove scrollbars, force 100vw × 100vh layout with touch-optimised controls.
- Voice interaction — Web Speech API for voice questions. Add mic button to Yantra chat in classroom mode.
- QR code generator for current lesson URL — display on smartboard so students can scan and join.
- Offline fallback — cache current lesson content via Service Worker (Workbox, already a Next.js option).

07 — Certificate & Portfolio System

- New table: student_certificates (id, user_id, skill_key, issued_at, verification_hash).
- Certificate issued when skill mastery >= 85% across quiz + void. Auto-triggered by skill engine.
- Certificate page: /certificate/:id — public, no auth, shows learner name, skill, date, QR verify link.
- Verification hash: SHA-256(user_id + skill_key + issued_at) stored in table. QR links to /verify/:hash.
- PDF download — reportlab on the server generates a styled PDF certificate. Route: /api/certificate/:id/pdf.
- Portfolio page: /dashboard/portfolio — lists all earned certificates with share buttons.
- Share to LinkedIn URL pre-filled with certificate URL and skill name.

08 — Git Workflow & Team Process

As shown in the Discord announcement, the team follows strict branch-based workflow. This section formalises the engineering process for 5 contributors.

Rule	Detail

Never commit to main	main is protected. All work goes through PRs.
Branch naming	feature/python-void, fix/mobile-nav-overflow, perf/image-optimisation
PR size	< 400 lines changed per PR. Break large features into sub-PRs.
Review	Krish reviews all PRs. Assign @krish as reviewer on every PR.
Pull before work	git pull origin main before starting any session.
Commit messages	feat: add Pyodide runner / fix: chat panel z-index / perf: lazy-load KaTeX
Deployments channel	Post Vercel preview URL in #deployments after every PR merge.
Bugs channel	Log every reproducible bug in #bugs with: page, steps to reproduce, screenshot.

Area	Priority	Suggested First Assignee
Performance audit + Lighthouse	P0 P0	Rishi or HIT HIT
Responsiveness fixes (mobile)	P1 P1	DMonkey451 or TG Krish
Python Void (Monaco + Pyodide)	P1 P2	Rishi TG HIT or
Multi-LLM orchestrator setup	P2 P3	DMonkey451 TG or Rishi
Skill engine DB tables + API		
Algorithm Void		
Smartboard classroom route		
Certificate system		

09 — New Routes & API Endpoints to Build

Route / Endpoint	Type Page	Description
/dashboard/void/python	Page Page	Python Void — Monaco + Pyodide + AI feedback
/dashboard/void/algorithm	Public Page	Algorithm Void — SVG step visualiser
/dashboard/portfolio	Public Page	Student portfolio — earned certificates list
/certificate/:id	Public Page	Certificate display + QR verification
/verify/:hash	Protected Page	Certificate verification endpoint
/classroom	POST POST	Smartboard classroom view
/classroom/teacher	POST GET	Teacher dashboard — class progress grid
/api/void/feedback	POST	AI code feedback — Claude Haiku
/api/quiz/grade		Grade quiz answers, update mastery
/api/skill/update		Recalculate skill mastery after session
/api/certificate/:id/pdf		Generate + download PDF certificate
/api/search		Web search synthesis via Gemini + Serper

10 — Environment Variables & Infrastructure

• Add ANTHROPIC_API_KEY to .env.example — required for Claude Haiku (Void feedback) and Claude Sonnet (deep explanations).
• Add SERPER_API_KEY to .env.example — 2500 free queries/month for web search tool.
• Add UPSTASH_REDIS_REST_URL + UPSTASH_REDIS_REST_TOKEN — rate limiting per model tier (free tier).
• Verify GEMINI_API_KEY is set in Vercel production environment — confirm in #deployments.
• Run the new SQL migrations for: student_skill_mastery, student_void_sessions, student_quiz_results, yantra_challenges, yantra_quiz_bank, student_certificates.
• Add @next/bundle-analyzer to devDependencies for bundle size audit (npm run analyze).
• Set up Vercel Analytics (free tier) — add to app/layout.tsx for real user metrics.

11 — Sprint Timeline (Indicative)

Sprint	Duration	Deliverables
Sprint 0 — Setup + Perf + Python Void	10 days	Git workflow enforced, Lighthouse baseline, .env updated, DB migrations run, bundle analyzer installe
Sprint 1 — Skill Engine + Algo 1V0o	10 days	All P0 performance fixes, Python Void live at /dashboard/void/python with 5 challenges
Sprint 2 — Multi-LLM + Smartb1o0a	10 days	Skill mastery DB live, dashboard cards dynamic, Algorithm Void live
Sprint 3 — Certificates + Portfo7l	10 days	Orchestrator routing Haiku/Sonnet/Flash, Classroom mode route live
Sprint 4 — Polish + Launch pre7p	7 days	Certificate system, PDF generation, portfolio page
		Final responsiveness audit, Lighthouse re-run, demo video, pilot school outreach

12 — Explicitly Out of Scope (Post-Launch)

Do not build these yet. They distract from the core learning loop.

• Custom fine-tuned model — wait until 1000+ real user sessions generate training data.
• Mobile PWA / React Native app — web must be solid first.
• Community Voids (student sharing + remixing) — requires moderation layer.
• LMS integrations (Moodle, Google Classroom) — Phase 3.
• Hiring partner API / job board — Phase 3.
• Real-time collaborative editing via WebRTC + Yjs — too complex, too expensive.
• Custom analytics dashboard beyond Vercel Analytics — not yet needed.